Linked List Construction

AlgoExpert · Medium · Linked Lists

The problem

- Doubly linked list with head/tail supporting setHead, setTail, insertBefore, insertAfter, insertAtPosition (1-based), removeNodesWithValue, remove and containsNodeWithValue.
- Inserting a node already in the list moves it.

Intuition

- A doubly linked list keeps head and tail pointers so most operations rewire a few links.

Brute force vs optimal

- Singly linked → removing needs the previous node (O(n))
- Doubly linked: O(1) insert/remove given a node
- Searching is still O(n)

Approach

- 1. Remove a node from its spot before reinserting
- 2. Update head and tail on the edges
- 3. Walk for position and value ops

Worked example




```
setHead(1)
```

setHead(1)

Worked example




```
setTail(4)
```

setTail(4)

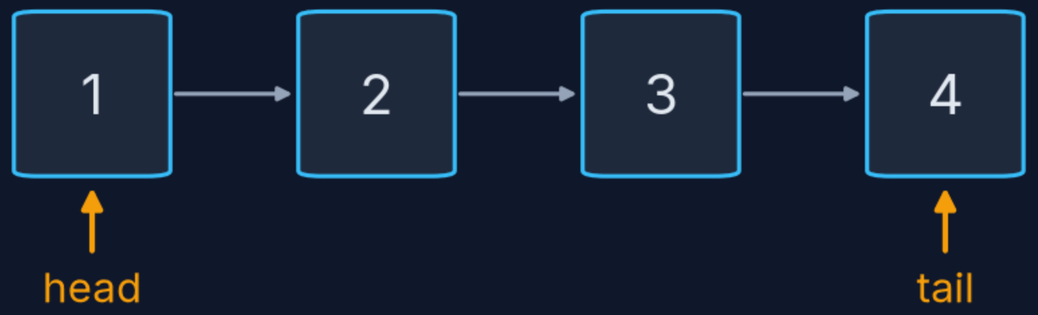
Worked example




```
insertBefore(4, 3)
```

insertBefore(4, 3)

Worked example




```
insertAtPosition(2, 2)
```

insertAtPosition(2, 2)

Worked example




```
removeNodesWithValue(3)
```

removeNodesWithValue(3)

Java solution (1/4)

```java
public class LinkedListConstruction {
    public static class Node {
        public int value;
        public Node prev, next;
        public Node(int value) { this.value = value; }
    }
```

Java solution (2/4)

```java
public static class DoublyLinkedList {
    public Node head, tail;

    public void setHead(Node n) {
        if (head == null) { head = tail = n; return; }
        insertBefore(head, n);
    }

    public void setTail(Node n) {
        if (tail == null) { setHead(n); return; }
        insertAfter(tail, n);
    }

    public void insertBefore(Node node, Node ins) {
        if (ins == head && ins == tail) return;
        remove(ins);
        ins.prev = node.prev;
        ins.next = node;
        if (node.prev == null) head = ins; else node.prev.next = ins;
        node.prev = ins;
    }
```

Java solution (3/4)

```
    public void insertAfter(Node node, Node ins) {
        if (ins == head && ins == tail) return;
        remove(ins);
        ins.prev = node;
        ins.next = node.next;
        if (node.next == null) tail = ins; else node.next.prev = ins;
        node.next = ins;
    }

    public void insertAtPosition(int pos, Node ins) {
        if (pos == 1) { setHead(ins); return; }
        Node cur = head;
        for (int p = 1; cur != null && p < pos; p++) cur = cur.next;
        if (cur != null) insertBefore(cur, ins); else setTail(ins);
    }

    public void removeNodesWithValue(int v) {
        Node cur = head;
        while (cur != null) { Node nx = cur.next; if (cur.value == v) remove(cur); cur = nx; }
    }
```

Java solution (4/4)

```
        public void remove(Node n) {
            if (n == head) head = head.next;
            if (n == tail) tail = tail.prev;
            if (n.prev != null) n.prev.next = n.next;
            if (n.next != null) n.next.prev = n.prev;
            n.prev = n.next = null;
        }

        public boolean containsNodeWithValue(int v) {
            for (Node c = head; c != null; c = c.next) if (c.value == v) return true;
            return false;
        }

        public java.util.List<Integer> values() {
            java.util.List<Integer> out = new java.util.ArrayList<>();
            for (Node c = head; c != null; c = c.next) out.add(c.value);
            return out;
        }
    }
}
```

Complexity

- Time: O(1) for head/tail/insertBefore/After/remove, O(n) for position/value ops
- Space: O(1)

Edge cases & pitfalls

- Inserting a node already in the list: remove it first
- Update head/tail when touching the ends
- Clear removed node's pointers

Interview tips & follow-ups

- Draw pointer changes before coding
- Used in LRU cache

Recap

- A doubly linked list keeps head and tail pointers so most operations rewire a few links.
- Complexity: O(1) for head/tail/insertBefore/After/remove, O(n) for position/value ops